NOVA-KV Part A — Baseline, Lock Profiling and Storage Layout

12 October 2026 · code `fe9ba2d` · all numbers from `results/part_a/`

1. What was built

`KVStore` (`put`/`get`/`erase`, Appendix A) is a hash table written from scratch. The key is mixed with a 64-bit finalizer; the low 8 bits pick one of 256 partitions, each a flat open-addressed array probed linearly (24-byte slots: key, value reference, state). Erase leaves a tombstone; a partition is rebuilt into a private array when live + tombstone slots would exceed 70 % or tombstones reach 20 %. Values are not stored in the index: a slot holds `(chunk_id, offset, length)` into an append-only arena of 64 KiB chunks, so values may be any length. `get` copies the bytes into the caller's buffer before returning.

The three required variants are the *same source files* compiled with a different lock type: `single_thread` (no lock), `global_lock` (one `std::mutex` held for every operation) and `rwlock` (`std::shared_mutex`, shared for `get`, exclusive for `put`/`erase`). Correctness is covered by 27 automated tests per build mode, including a 570,000-operation randomized comparison against a reference model, allocation-failure injection, and a one-writer / four-reader torn-record test that also runs under ThreadSanitizer.

2. How it was measured

Machine. AMD Ryzen AI 7 350 laptop: 8 cores / 16 logical CPUs, of which four cores reach 5.09 GHz and four 3.51 GHz; 30 GiB RAM; Ubuntu 26.04, Linux 7.0; GCC 15.2, `-O3 -g -DNDEBUG -fno-omit-frame-pointer`. On AC power, `performance` governor for the whole campaign (`conditions.log`).

Two tracks, kept apart. The supplied `workload_gen.cpp` loads all 100,000 trades *before* it starts any reader, so it cannot measure readers against an active writer. It is run unmodified through the three drivers and reported as **preload-only**. The profiled numbers come from our own harness (`benchmarks/concurrent_bench.cpp`) with the same records, keys, hot set and 80 % hot / 20 % random read mix, in which readers and one writer thread start together and overlap for the whole 3 s window (after a 0.5 s warm-up; loading excluded). The writer overwrites existing trades, either as fast as the lock allows ("unthrottled") or paced at 10,000 updates/s. Every run checks that the writer completed updates inside the window and that every read returned the right record.

Method. Threads are pinned: readers one per physical core, fast cores first; the writer on the SMT sibling of the last core. Each `get` is timed with two clock reads (as the supplied generator does) into a fixed-memory histogram; the maximum is exact. 5 separate processes per configuration; tables give the median and the min–max range; no run is discarded. `perf stat` wraps each of those processes. Flame graphs: `perf record -F 1999 --call-graph fp` on cycles, `perf script`, `stackcollapse-perf.pl`, `flamegraph.pl`. Cache-line contention: `perf c2c`.

3. Three synchronization policies

Single thread, no lock, no writer: **18.95 M gets/s** (18.74–18.99), p50 0.03 µs, p99 0.06 µs. With the same thread also doing one update per 100 reads: 17.96 M gets/s. This is the cost of the data structure plus the two clock reads, with no synchronization at all.

Table 1: reads with an active writer (R = reader threads). "max µs" is the longest single `get` in any of the 5 runs. The last column is the unmodified generator: no writer running.

variant	R	read M/s	min–max	writes/s	p50 µs	p99 µs	p99.9 µs	max µs	paced: read M/s	paced: writes/s	paced: p99 µs	preload-only M/s
global_lock	1	5.57	5.25–5.81	2,909,886	0.05	2.72	4.35	132	17.06	10,000	0.07	15.60
global_lock	2	6.71	6.62–6.99	2,135,953	0.05	4.22	8.45	3,760	17.79	10,000	0.28	15.82
global_lock	4	8.70	8.23–8.83	1,545,131	0.06	4.80	9.47	2,637	20.12	10,000	4.00	16.04
global_lock	8	7.89	7.74–7.93	601,952	0.22	8.96	16.00	667	11.47	10,000	7.30	9.96
rwlock	1	0.16	0.15–0.16	157,235	5.95	9.22	29.70	2,944	16.79	10,000	0.08	15.46
rwlock	2	0.58	0.52–0.84	146,647	5.44	15.49	29.70	3,466	24.28	10,000	0.11	18.99
rwlock	4	1.65	1.48–1.69	132,901	0.10	8.32	29.70	3,824	34.55	10,000	0.24	25.12
rwlock	8	21.23	21.08–21.27	1,202	0.40	0.70	1.17	5,447	21.30	1,098	0.69	19.17

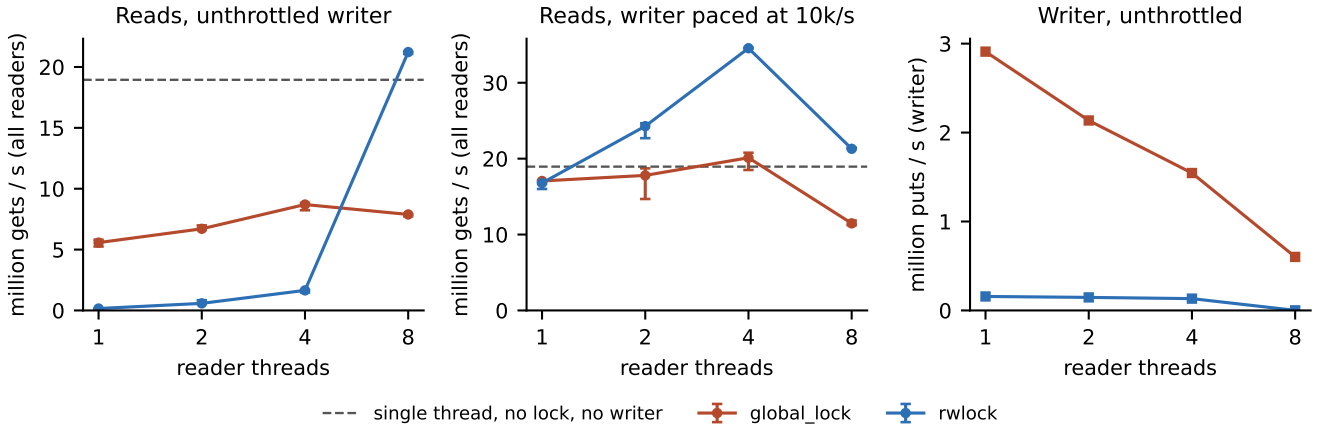


Figure 1: Read throughput against reader threads (median, bars = min–max of 5 runs), and writer throughput in the unthrottled case.

Where each lock stops scaling. `global_lock` gains little from 1 to 4 readers (5.57 → 8.70 M/s unthrottled; 17.06 → 20.12 M/s paced) and *loses* throughput at 8 (7.89 and 11.47 M/s): it stops scaling at 4 readers. `rwlock` with a rare writer scales to 4 readers (16.79 → 34.55 M/s, 2.06×) and then falls to 21.30 M/s at 8: it also stops at 4. The same shape appears in the preload-only runs (25.12 → 19.17 M/s). The drop from 4 to 8 is not explained by the slower cores: four more readers at 0.69× the clock rate would add throughput, not remove 38 % of it.

`rwlock` with a busy writer has two regimes. With 1–4 readers and an unthrottled writer, reads collapse to 0.16–1.65 M/s, far below the mutex, and the median `get` takes 5.95 µs at one reader: reader and writer keep handing the lock to each other through the kernel. At 8 readers the readers overlap continuously, hold the lock shared without a gap, and the **writer starves**: 1,202 updates/s, and only 1,098/s when asked for 10,000/s. glibc's `shared_mutex` prefers readers; throughput looks good (21.2 M/s) only because writes have almost stopped.

Tail latency. Under the mutex, p99 grows from 2.72 µs to 8.96 µs and p99.9 to 16 µs with 8 readers, against 0.06 µs single-threaded. Maxima of 0.1–5.4 ms occur in most configurations, including ones with little contention; they are consistent with a reader being descheduled, and we did not attribute them further.

4. Where the lock-based versions lose time

Table 2: evidence per configuration, unthrottled writer. `perf stat` columns are medians of 5 runs (counters cover the whole process). "lock acq./rel." are the shares of sampled CPU cycles in stacks inside the lock or unlock call (from the folded flame-graph stacks); "in kernel" is the part of those spent in the kernel, which for these binaries is the futex wait/wake path. The last three columns are from `perf c2c`: loads that hit a cache line held Modified by another core (HITM), and where those lines are.

variant	R	CPUs used	IPC	cache-miss %	cycles/op	ctx-sw/s	lock acq. %	lock rel. %	in kernel %	store %	harness %	HITM loads	on lock line %	on kernel lines %
global_lock	1	1.76	1.37	19	935	102,450	24	31	34	9.3	32	302	29	47
global_lock	2	2.41	1.20	15	1,185	260,380	38	32	46	8.9	19	640	29	62
global_lock	4	3.88	1.02	15	1,639	534,025	46	37	56	4.6	12	1,663	26	72
global_lock	8	7.25	0.63	16	3,413	727,297	44	46	77	2.1	8	3,482	12	87
rwlock	1	0.91	1.93	4	5,840	314,086	59	25	74	5.3	11	100	22	71
rwlock	2	1.25	1.78	6	3,806	437,035	63	22	72	5.6	9	263	19	79
rwlock	4	2.03	1.38	7	3,094	662,531	66	21	62	4.9	7	559	45	54
rwlock	8	7.90	0.28	41	1,850	10,610	26	36	1	2.1	36	1,093	70	30

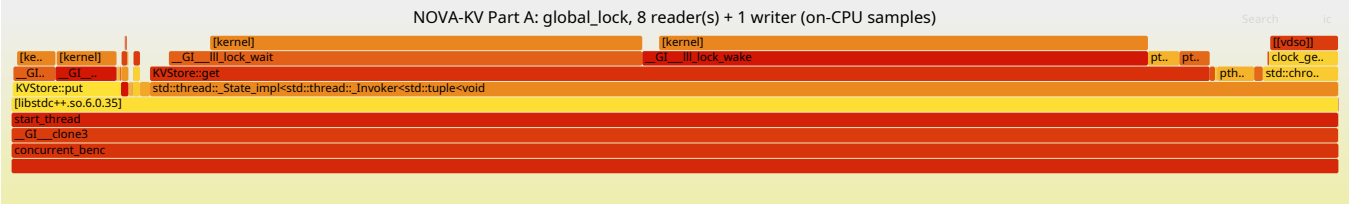


Figure 2: `global_lock`, 8 readers + writer. The two wide towers are `KVStore::get` → `__lll_lock_wait` and `__lll_lock_wake` → kernel.

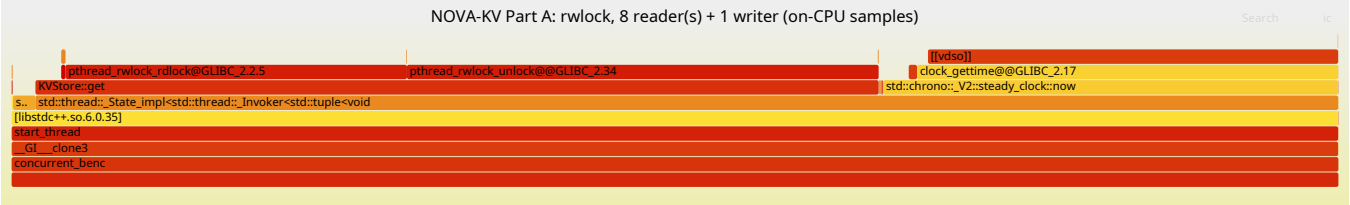


Figure 3: `rwlock`, 8 readers + writer. No kernel tower: the width is in `pthread_rwlock_unlock/rdlock` themselves and in the clock read that follows.

`global_lock` at 8 readers loses its time acquiring and handing over the lock. 90 % of CPU cycles are in stacks under `mutex::lock/unlock` and 77 % of all cycles are in the kernel below `__lll_lock_wait` and `__lll_lock_wake` (Figure 2). The hash probe and copy account for 2.1 %. `perf stat` agrees: 727,000 context switches per second, cycles per completed

operation up from 935 to 3,413, IPC down from 1.37 to 0.63. So threads really are waiting, and paying a system call to sleep and another to be woken. The cache-miss *ratio* stays flat (15–19 %), which by itself would not suggest bouncing; `perf c2c` shows why: 87 % of HITM loads are on kernel cache lines (the futex queue the sleepers share) and 12 % on the mutex's own line. The contention has moved into the kernel; there is still line bouncing, but as a by-product of blocking.

`rwlock` at 8 readers loses its time to cache-line bouncing, without waiting. Only 1 % of cycles are in the kernel and context switches fall to 10,610/s: nobody sleeps. Yet a `get` takes 0.40 µs at the median, 13× the single-threaded 0.03 µs, IPC is 0.28 and the cache-miss ratio jumps from 4–7 % to 41 %. 62 % of cycles are inside `pthread_rwlock_rdlock/unlock` in user space (Figure 3). `perf c2c` identifies the cause directly: 70 % of HITM loads fall on the one cache line that holds the store's `shared_mutex` (the harness prints the lock's address; the report attributes those loads to `pthread_rwlock_unlock` and `_rdlock`). Every shared acquire and release is an atomic update of the reader count in that line, so the line is pulled from core to core on every `get` although no reader conflicts with another.

Two cautions. First, these are on-CPU profiles: a blocked thread is not sampled, so in the 1–4 reader `rwlock` rows the dominant cost (sleeping) shows up as low "CPUs used" (0.91–2.03) and low throughput, not as width. Second, in Figure 3 the clock read takes 34 % of cycles although it does constant work; we believe it absorbs time waiting for the preceding lock-word update to complete, but that attribution is an inference, not a measurement.

5. Storage layout design note

Question. How should one record's fields be laid out for this workload: point gets that return a whole 32-byte `Trade` (`trade_id`, `symbol[8]`, `price`, `ts`), and the Part C join, which for each trade probes an index on `symbol` and then fetches a whole `Instrument` row?

Candidates. *NSM* stores each row contiguously (one array of 32-byte rows). *DSM* stores each field in its own array. *PAX* groups 128 records into a 4096-byte page and stores the fields column-wise inside the page (four 1024-byte mini-pages).

Microbenchmark (`benchmarks/layout_bench.cpp`). All three layouts are built from the same data and driven with the same pre-generated access sequences, addressed by record number so that only the layout differs (the hash index would be identical). A checksum of every value read must match across layouts or the run fails. Two table sizes: 100,000 records (3 MiB, the workload's size) and 8,000,000 (244 MiB, far beyond the 16 MiB L3). Patterns: `row_chain` reads whole rows where each position comes from the previous row (true per-get latency); `row_uniform` and `row_hot` read whole rows at independent positions (throughput; `row_hot` is the workload's 80/20 mix); `sym_price` and `field_price` read two and one field; `scan_price` sums one field over the table. 7 repetitions interleaved across layouts on one pinned fast core; medians shown, ns per access.

records	pattern	NSM ns	DSM ns	PAX ns	DSM/NSM	PAX/NSM
100,000	row_chain	9.87	10.39	10.77	1.05×	1.09×
100,000	row_uniform	1.34	2.05	1.95	1.53×	1.45×
100,000	row_hot	1.33	1.73	1.67	1.30×	1.25×
100,000	sym_price	0.92	0.92	0.92	1.00×	1.00×
100,000	field_price	0.73	0.41	0.56	0.56×	0.76×
100,000	scan_price	0.25	0.19	0.32	0.75×	1.28×
8,000,000	row_chain	118.10	124.64	128.68	1.06×	1.09×
8,000,000	row_uniform	8.98	16.42	14.26	1.83×	1.59×
8,000,000	row_hot	5.49	13.35	11.40	2.43×	2.08×
8,000,000	sym_price	6.37	5.61	7.49	0.88×	1.18×
8,000,000	field_price	3.95	2.96	5.73	0.75×	1.45×
8,000,000	scan_price	0.63	0.21	1.41	0.32×	2.22×

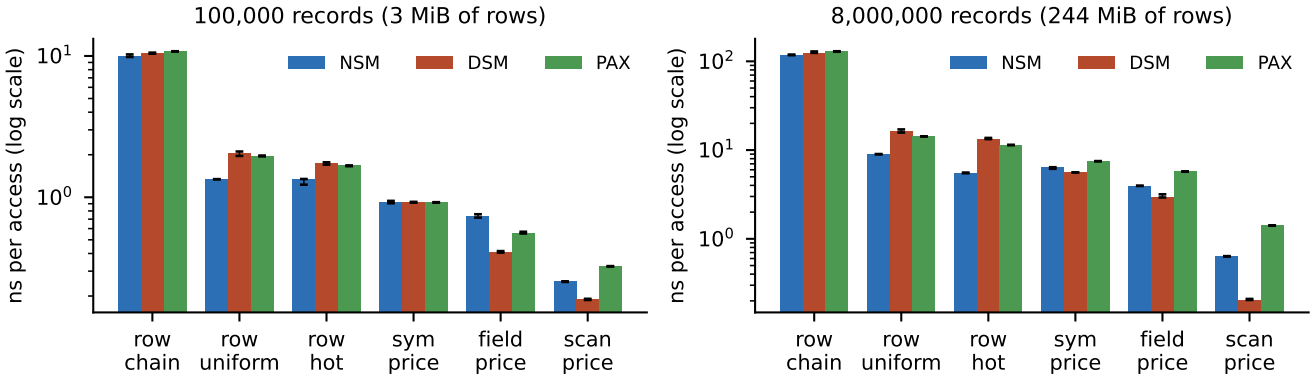


Figure 4: Layout microbenchmark, median with min–max bars; log scale.

What the numbers say.

- *Whole-row point gets favour NSM, increasingly so as the table outgrows the cache.* At 8 M records NSM needs 8.98 ns per row against 16.42 (DSM) and 14.26 (PAX); with the workload's hot/cold mix 5.49 against 13.35 and 11.40, i.e. DSM is 2.4× and PAX 2.1× slower. At 100,000 records the gap is 1.3–1.5×. The reason is visible in the counters: a row costs 1.25 cache-miss events in NSM and 5.18 / 4.65 in DSM / PAX (8 M records, `row_uniform`), for about the same instructions (19 vs 22–24). One row is one cache line in NSM and four lines elsewhere.
- *As pure latency the gap is small.* With dependent accesses (`row_chain`) NSM is only 5–9 % ahead (118 vs 125 and 129 ns at 8 M): the CPU fetches the four column lines in parallel, so the wait is roughly one miss either way. DSM and PAX cost memory bandwidth and cache capacity more than they cost time per isolated get, which matters once eight readers compete for the same caches.
- *PAX recovers part of DSM's loss* on rows (its four lines share one 4 KiB page, so one TLB entry instead of four), but never reaches NSM.

- *Columns win when few fields are read.* A single field is fastest in DSM (2.96 vs 3.95 ns at 8 M); a full column scan is 3× faster in DSM (0.21 vs 0.63 ns per value). `symbol` + `price` is a tie at 100,000 records and 12 % better in DSM at 8 M. Our PAX scan is the slowest (1.41 ns) because the benchmark computes page and slot per access; a scan written page by page would do better, so this one number reflects our implementation more than PAX itself.

Decision: NSM. The operations this store performs through its API are all whole-value: `get` returns a complete row, and the join fetches complete `Instrument` rows; only its probe key is a single field, and that comes out of a `Trade` the join has already fetched whole. Appendix A has no way to ask for one column, so the cases where DSM wins (single-field reads, scans) cannot occur through this interface, while the case where it loses by up to 2.4× is the hot path. PAX would be the choice if analytic scans over one field were added next to the point gets; nothing in Parts A–D does that. The generic store therefore keeps each value as one contiguous run of bytes in the arena, which is NSM for whatever record the caller stores.

Scope of this claim. The benchmark measures the layout in isolation with direct addressing, single-threaded, for one 32-byte schema on one machine. It does not show the effect inside the store, where the hash probe and the lock dominate (§4). The cache-miss counter is perf's generic event, whose exact cache level is CPU-specific; we use it only to compare layouts.

6. Limitations and what is not shown

- The supplied generator's numbers are preload-only; all active-writer results come from our own harness, whose writer rate (unthrottled or 10,000/s) is our choice and strongly affects the outcome, as Table 1 shows.
- Timing every `get` with two clock reads adds a fixed cost to each operation in every configuration (it is why the single-thread figure is 19 M/s and not higher).
- Heterogeneous cores and SMT: readers 5–8 run on slower cores and the eighth shares a core with the writer, so 4 → 8 is not a like-for-like doubling.
- Kernel symbols are hidden on this machine (`kptr_restrict = 1`): kernel time is attributed by the user-space caller only. `perf c2c` sampled a few hundred to a few thousand HITM loads per run; the shares in Table 2 come from one 5 s run each.
- Value bytes are not cache-line aligned in the arena: about half of the 32-byte rows straddle two lines. This is a known inefficiency of the current store, left unchanged here.
- One machine, one compiler, 3 s windows, 5 repetitions. Raw files for every number: `results/part_a/` (`concurrent/`, `reference/`, `layout/`, `flamegraphs/`, `c2c/`, `summary/`). All 8 flame graphs are in `results/part_a/flamegraphs/*.svg`.